Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Açık adresleme (open addressing) tekrarı ve analizi
  - Çakışmalar (collision) ya zincirleme (separate chaining) ile ya da açık adreslemeyle çözülür. Açık adreslemede hash fonksiyonu ikinci parametre olarak deneme (probe) numarasını alır; bütün elemanlar tek bir tabloda tutulur. Deneme yöntemleri: doğrusal deneme (linear probing), karesel deneme (quadratic probing), çift hashing (double hashing).
  - Slot sayısı m, eleman sayısı n olmak üzere tablo elemandan büyük tasarlanır; yük faktörü $\alpha = n/m < 1$ olur.
  - Teorem (ekrandan, 00:10:00): düzgün hashing (uniform hashing) varsayımıyla, $\alpha < 1$ olan bir tabloda başarısız aramadaki beklenen deneme sayısı en fazla $1/(1-\alpha)$'dır.
- Teoremin ispatı (ekrandan, 00:20:00)
  - X: başarısız bir aramada yapılan deneme sayısı. A_i: i. denemenin dolu (occupied) bir slota denk gelmesi olayı.
  - İlk denemenin dolu slota gelme olasılığı n/m, ikincisinin $(n-1)/(m-1)$, üçüncüsünün $(n-2)/(m-2)$ olur.
  - $\Pr\{X \geq i\} = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+2}{m-i+2}$. $m > n$ olduğundan $\frac{n-j}{m-j} \leq \frac{n}{m}$; bu yüzden $\Pr\{X \geq i\} \leq \alpha^{i-1}$.
  - $E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}$.
  - Yorum: $\alpha = 0{,}5$ ise en fazla 2 deneme, $\alpha = 0{,}9$ ise 10 deneme. Tablo doldukça açık adresleme yavaşlar.
- Yeniden hashing (rehashing) (ekrandan, 00:23:40)
  - Tablo çok dolunca iki kat büyüklükte yeni bir tablo açılır; silinmemiş bütün elemanlar yeni hash fonksiyonuyla yeni tabloya taşınır.
  - Örnek: $h(x) = x \bmod 7$, doğrusal denemeyle 13, 15, 6, 24 yerleşmiş (6→0, 15→1, 24→3, 13→6). 23 eklenince (slot 2) tablo %70'in üstünde dolu olur. Eski boyutun iki katı 14'ten sonra gelen asal 17 seçilir, yeni fonksiyon $h(x) = x \bmod 17$ olur.
  - Pahalı bir işlemdir (n eleman taşınır) ama seyrek yapılır. Etkileşimli uygulamada denk gelen kullanıcı yavaşlama yaşar.
- Evrensel hashing (universal hashing) (ekrandan, 00:27:35)
  - Her sabit hash fonksiyonu için çakışma doğuracak bir anahtar dizisi üretilebilir; kötü niyetli biri bunu kullanabilir. Çözüm: programın başında hash fonksiyonunu bir aileden rastgele seçmek. Kodu gören biri bile hangi fonksiyonun seçildiğini bilemez.
  - Tanım: H, U evrenini $\{0, 1, \ldots, m-1\}$'e eşleyen sonlu bir fonksiyon kümesidir. Farklı her k, l çifti için $h(k) = h(l)$ olan fonksiyon sayısı en fazla $|H|/m$ ise H evrenseldir. Yani rastgele seçilen bir h için çakışma olasılığı en fazla $1/m$'dir.
  - Hoca k çift için genellemeyi de söyledi: $h(x_1) = h(y_1), \ldots, h(x_k) = h(y_k)$ olan fonksiyon sayısı en fazla $|H|/m^k$.
- Asal sayılarla evrensel aile (ekrandan, 00:30:00 ve 00:32:35)
  - Bütün anahtarlar 0 ile $p-1$ arasında kalacak kadar büyük bir asal p seçilir; $p > m$.
  - $H = \{h_{a,b}(x) = ((ax+b) \bmod p) \bmod m\}$, $1 \leq a \leq p-1$, $0 \leq b \leq p-1$. Ailede $p(p-1)$ fonksiyon vardır.
  - Örnek ($p = 17$, $m = 6$): $h_{5,7}(3) = (15+7) \bmod 17 = 5$, $5 \bmod 6 = 5$; $h_{3,4}(28) = (84+4) \bmod 17 = 3$, $3 \bmod 6 = 3$; $h_{10,2}(3) = 32 \bmod 17 = 15$, $15 \bmod 6 = 3$.

- Matris yöntemi (matrix method) (ekrandan, 00:35:10)
  - Anahtarlar u bit uzunluğundadır; tablo boyutu $m = 2^b$ olarak seçilir. Rastgele $b \times u$ boyutlu bir 0/1 matrisi h seçilir ve $h(x) = hx$ (toplama mod 2) tanımlanır. Her matris bir hash fonksiyonudur.
  - Örnek: satırları 1000, 0111, 1110 olan matris ile $x = (1,0,1,0)^T$ çarpılınca $h(x) = (1,1,0)^T$.
  - Ailede 2^{ub} fonksiyon vardır (her eleman 0 ya da 1). Örnek: 32 bitlik IP adresleri, 256 slotlu tablo (2^8): 8×32 matris, $2^{8 \cdot 32}$ fonksiyon.
- Matris yönteminin evrensel olduğunun ispatı (ekrandan, 00:45:30)
  - $x \neq y$ ise en az bir i bitinde farklıdırlar: $x_i = 0$, $y_i = 1$ alınır. h', h'nin i. sütunu çıkarılmış hâlidir.
  - $x_i = 0$ olduğu için $h(x) = h'(x)$; $y_i = 1$ olduğu için $h(y) = h'(y) + h_i$ (h_i: i. sütun).
  - Diğer bitler aynı kabul edilince $h'(x) = h'(y)$; eşitlik yalnız h_i sıfır vektörü olduğunda sağlanır. h_i'nin 2^b olası değeri vardır, olasılık $1/2^b = 1/m$.
- n anahtarda en az bir çakışma olasılığı (ekrandan, 01:06:45)
  - Anahtar çifti sayısı $\binom{n}{2} = n(n-1)/2$. Bir çiftin çakışma olasılığı $1/m$, çakışmama olasılığı $1 - 1/m$.
  - $\Pr(\#\text{çakışma} \geq 1) = 1 - \Pr(\#\text{çakışma} = 0) = 1 - \left(1 - \frac{1}{m}\right)^{n(n-1)/2} = 1 - \left(\frac{m-1}{m}\right)^{n(n-1)/2}$.
  - Büyük tablo örneği (ekrandan, 01:10:00): $n = 1000$, $m = 100000$ için olasılık $0{,}9932$. 100 rastgele fonksiyonun hepsinde çakışma olasılığı $0{,}9932^{100} = 0{,}507$; yani birinin çakışmasız olma şansı yaklaşık %50. Yük faktörü %1 iken bile durum budur; mükemmel hash fonksiyonu seçmek için iyi bir yol değildir.
- Mükemmel hashing (perfect hashing)
  - Amaç en kötü durumda $O(1)$ erişimdir. Anahtar kümesi statik olduğunda kullanılır (CD-ROM'a yazılan veri, derleyicide ayrılmış kelimeler).
  - İki seviyeli hashing yapılır, her seviyede evrensel hashing kullanılır. Bağlı liste yerine her çakışan j slotu için ikinci seviye küçük bir tablo ve ona ait h_j fonksiyonu tutulur.
  - İkinci tabloda çakışma olmaması için boyutu $m_j = n_j^2$ seçilir (n_j: j slotuna düşen anahtar sayısı). Birinci fonksiyon iyi seçilirse toplam yer $O(n)$ kalır.
  - Adımlar (ekrandan, 01:19:05): (1) a, b rastgele seçilir, p anahtarlardan büyük seçilir; (2) her anahtarın ev hücresi $h(k)$ bulunur, sayılır; (3) $\sum_{j=0}^{m-1} n_j^2 < 2n$ kontrol edilir; (4) alt tablolar yeni hash fonksiyonlarıyla doldurulur.
  - Örnek: $K = \{8, 22, 36, 75, 61, 13, 84, 58\}$, $p = 87$, $a = 64$, $b = 5$, $m = 8$. Slotlar: 1: 61; 2: 8, 84; 4: 75; 5: 22; 6: 13; 7: 36, 58. $\sum n_j^2 = 12 < 16$. Slot 2 için $m_j = 4, a_j = 82, b_j = 53$ (84→0, 8→1); slot 7 için $m_j = 4, a_j = 10, b_j = 54$ (58→1, 36→2) (ekrandan, 01:20:00).
- Mükemmel hashingle ilgili iki teorem
  - Toplar ve kutular (balls and bins): n top $m = n^2$ kutuya atılırsa hiçbir kutuda birden fazla top olmama olasılığı en az $1/2$'dir. İspat: beklenen çakışma sayısı $\binom{n}{2}\frac{1}{n^2} = \frac{n-1}{2n} < \frac{1}{2}$.
  - $m = n$ slotlu birinci tabloya n anahtar konursa $E\left[\sum_{j=0}^{m-1} n_j^2\right] < 2n$ (ekrandan, 01:36:25). İspat $a^2 = a + 2\binom{a}{2}$ eşitliğiyle: $E[\sum n_j^2] = E[\sum n_j] + 2E[\sum \binom{n_j}{2}] = n + 2\binom{n}{2}\frac{1}{m} = n + 2 \cdot \frac{n-1}{2} = 2n - 1 < 2n$.
- Cuckoo hashing (ekrandan, 01:40:00)
  - Adını, yumurtasını başka kuşun yuvasına bırakıp oradaki yumurtayı atan guguk kuşundan alır. Amaç en kötü durumda $O(1)$ sorgu.
  - İki hash fonksiyonu (f, g) ve iki tablo vardır. Query(x): $T[f(x)]$ ve $T[g(x)]$ kontrol edilir. Delete(x): bulunursa silinir.
  - Insert(x): $T[f(x)]$ boşsa yerleşir. Doluysa x yine oraya konur, eski eleman y öbür seçeneğine atılır; o da doluysa süreç tekrar eder. $6 \log n$ atma (bump) sınırına gelinirse yeni f, g seçilir ve bütün tablo yeniden hashlenir.
  - Sorgu ve silme $O(1)$; eklemenin analizi derste yapılmadı.
- Cuckoo örneği (ekrandan, 02:00:00)

  - Hash değerleri (tablo 1, tablo 2): A: 0, 2; B: 0, 0; C: 1, 4; D: 1, 0; E: 3, 2; F: 3, 4.
  - Sonuç: tablo 1'de 0: A, 1: D, 3: F; tablo 2'de 0: B, 2: E, 4: C.
  - G (1, 2) eklenince elemanlar döngüye girer, G yine açıkta kalır; tablo yeniden hashlenmelidir.
- Hopscotch hashing (seksek)
  - Doğrusal denemenin kümelenme (clustering) sorunu vardır, ama bellekte yan yana duran veriyi birlikte çektiği için donanım açısından avantajlıdır. Hopscotch bu avantajı korur.
  - Donanıma göre bir komşuluk boyu H (en fazla uzaklık) belirlenir. Her eleman ev slotundan en fazla $H-1$ uzakta durur. Arama $h(x), h(x)+1, \ldots, h(x)+H-1$ slotlarına bakar: $O(1)$. Silme tembel silmeyle (lazy deletion) yapılır.
  - Ekleme: doğrusal denemeyle ilk boş slot bulunur; komşuluk dışındaysa boşluk, ev slotundan uzaklaşmayacak elemanlarla yer değiştirilerek geri taşınır. Taşınamıyorsa yeniden hashing gerekir. Yarı dolu tabloda başarısızlık olasılığı neredeyse sıfırdır.
  - Örnek (ekrandan, 02:10:00): $H = 4$; a1, b1, c3, d4, r3, s5, t7 (alt simge ev slotu). x'in ev slotu 1, ilk boş slot 8. Adaylar 5, 6, 7: r (ev 3) taşınamaz, s (ev 5) 8'e taşınır; boşluk 6'da. Sonra c (ev 3) 6'ya taşınır, boşluk 3'e gelir; x 3'e yerleşir.
  - Bitmap (hop) gösterimi: her slot için H bitlik dizi; k. bit 1 ise o slota ait bir eleman k uzaktadır.
  - Örnek (ekrandan, 02:16:00): H'nin ev slotu 9, ilk boş yer 13 (çok uzak). E (ev 8) taşınamaz; G (slot 11, ev 11) 13'e taşınır; H 11'e yerleşir, slot 9'un hop dizisi 1000'den 1010'a, slot 11'inki 0010'a döner.
- Genişletilebilir hashing (extendible hashing) (ekrandan, 02:20:00 ve 02:25:35)
  - Veri ana belleğe sığmıyorsa asıl ölçüt disk erişim sayısıdır. Çok ekleme ve silmede bile performansı yüksektir.
  - Kovalar (bucket) diskte durur. Dizin (directory) önek (prefix) bitlerine göre kovaları gösterir. Bir kovada bir disk bloğunun alabileceği kadar (M) eleman olur.
  - Örnek: 6 bitlik veri, 2 bitlik dizin (00, 01, 10, 11), kova başına en fazla 4 eleman. 10 kovası dolu (100000, 101000, 101100, 101110) iken 100100 eklenir; kova bölünür, dizin 3 bite çıkar: 100 kovasında 100000, 100100; 101 kovasında 101000, 101100, 101110. Diğer kovalara dokunulmaz; iki dizin girdisi aynı kovayı gösterebilir.
  - Ardından 000000 eklenince 00 kovası (4 eleman) bölünür ama dizin yeterli olduğu için 4 bite çıkılmaz.
  - Silme sonrası iki kardeş kovanın toplamı kapasiteye inerse birleştirilir; iki dizin girdisi aynı kovayı gösterir.
  - Dizin belleğe sığmasa bile işlemler en fazla iki disk erişimiyle yapılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Açık adreslemede doluluk arttıkça deneme sayısı hızla büyür
  - $\alpha = 0{,}5$ için en fazla 2, $\alpha = 0{,}9$ için 10 deneme; bir noktadan sonra başka çözüm (yeniden hashing ya da başka yöntem) gerekir.
- Evrensel hashingin amacı güvenliktir
  - Hash fonksiyonu rastgele seçildiği için kötü niyetli biri kodu görse bile en kötü durum dizisini oluşturamaz.
- Mükemmel hashingde $m_j = n_j^2$ şartı önemlidir
  - Hoca bu ilişkinin önemini ayrıca belirtti. Yer kaybı korkusunu ikinci teorem giderir: beklenen toplam alt tablo boyutu $2n$'den küçüktür.
- Cuckoo hashingde eleman önce ilk tercihine yerleşir
  - İkinci seçeneği boş olsa bile eleman ilk tercihine konur ve oradaki eleman atılır.
- İspatların amacı
  - Hoca formülleri kalem kağıtla hesaplamak için değil, yöntemlerin mantığını oturtmak ve akademik çalışmaya hazırlık için ispatladığını söyledi.

Kısa Tekrar Notları

- Açık adreslemede başarısız arama: $E[X] \leq 1/(1-\alpha)$.
- Yeniden hashing: iki kat boyutlu tablo, $2m$'den büyük ilk asal, yeni fonksiyon, bütün elemanlar taşınır.

- Evrensel aile: rastgele h için $\Pr[h(k) = h(l)] \leq 1/m$.
- Asal yöntem: $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$, $p(p-1)$ fonksiyon.
- Matris yöntemi: $b \times u$ rastgele 0/1 matris, $h(x) = hx \bmod 2$, $m = 2^b$, 2^{ub} fonksiyon, çakışma olasılığı $1/2^b$.
- n anahtarda en az bir çakışma: $1 - ((m-1)/m)^{n(n-1)/2}$.
- Mükemmel hashing: iki seviye, $m_j = n_j^2$, $\sum n_j^2 < 2n$ kontrolü, beklenen toplam yer $< 2n$.
- n top, n^2 kutu: çakışmasız olma olasılığı $\geq 1/2$.
- Cuckoo: iki tablo, iki fonksiyon, sorgu ve silme $O(1)$, $6 \log n$ atmadan sonra yeniden hashing.
- Hopscotch: komşuluk H, boşluk geri taşınır, hop bitmap'i, arama $O(1)$.
- Genişletilebilir hashing: önek bitli dizin, kova bölünmesi, gerekirse dizin ikiye katlanır, en fazla iki disk erişimi.

Detaylı Açıklamalar

- Açık adreslemenin analizinde her başarısız deneme bir sonraki denemeyi gerektirir. İlk deneme n/m olasılıkla dolu slota gelir. Bir slot ve bir eleman elendiği için ikinci deneme $(n-1)/(m-1)$ olasılıkla dolu slota gelir; bu böyle sürer. $X \geq i$ olması için ilk $i-1$ denemenin hepsinin dolu slota gelmesi gerekir; bu olayların koşullu olasılıkları çarpılır. Her çarpan n/m'den büyük olmadığı için çarpım α^{i-1} ile sınırlanır. Beklenen değer, $\Pr\{X \geq i\}$ terimlerinin toplamıdır ve geometrik seri $1/(1-\alpha)$'yı verir. Hoca ispatı Microsoft Whiteboard üzerinde yazdı.
- Yeniden hashing problemi geçici olarak çözer. Yeni tablo boyutu eski boyutun iki katından büyük ilk asal sayıdır (7 için 17). Bütün elemanlar tek tek taranıp yeni fonksiyonla yerleştirildiği için pahalıdır, ama sık yapılmaz.
- Ders sonrası not: Hoca "14'e yakın asal sayı arıyorum ve 17'yi seçiyorum" dedi. 14'e en yakın asal 13'tür; 17, 14'ten büyük ilk asaldır. Kural, eski boyutun iki katından büyük ilk asal olarak okunmalıdır.
- Evrensel hashing, sabit bir hash fonksiyonunun her zaman kötü bir girdiyle çökertilebileceği gözleminden doğar. Aileden rastgele seçim yapılır; tanımdaki $|H|/m$ sınırı, rastgele seçilmiş bir fonksiyonda iki farklı anahtarın çakışma olasılığının en fazla $1/m$ olmasıyla aynı şeydir. Asal yöntemde a ve b rastgele seçilir; en sondaki mod$m$ sonucu tablonun içinde tutar. Matris yönteminde rastgele ikili matris fonksiyonun kendisidir; çarpım mod 2 ile yapılır.
- Matris yönteminin ispatında x ve y yalnız bir bitte farklı kabul edilir (hoca bunu en kötü durum diye niteledi). O bit x'te 0 olduğu için ilgili sütun $h(x)$'e katkı vermez; y'de 1 olduğu için sütun $h(y)$'ye eklenir. İki değerin eşit olması için sütunun sıfır vektörü olması gerekir; 2^b olası sütundan yalnız biri bunu sağlar.
- Ders sonrası not: Hoca "daha fazla bit farklı olsaydı çakışma olasılığı düşerdi" dedi; doğrusu, $x \neq y$ olan her durumda olasılık tam $1/2^b$'dir. Diğer sütunlar sabitlendiğinde $h(x) = h(y)$ için h_i'nin tek bir belirli vektöre ($h'(x) + h'(y)$, mod 2) eşit olması gerekir; bu da $1/2^b$ olasılıktır.
- n anahtarda en az bir çakışma olasılığı hesabı, evrensel hashingin büyük tablolarda pratik olmadığını gösterir. 1000 anahtar ve 100000 slotta (yük faktörü %1) bile çakışma olasılığı yaklaşık %99'dur. Olasılığın %50'ye inmesi için tablonun 700 bin civarına çıkması gerekir. 300 rastgele fonksiyon denenirse hepsinin çakışma olasılığı yaklaşık %13, en az birinin çakışmasız olma olasılığı yaklaşık %87 olur.
- Mükemmel hashingde birinci seviyede çakışma olabilir; önemli olan ikinci seviye tablolarda çakışma olmamasıdır. Birinci fonksiyon, $\sum n_j^2 < 2n$ şartını sağlayana kadar yeniden seçilir. Sonra her alt tablo için $m_j = n_j^2$ boyutlu tablo ve rastgele a_j, b_j ile ikinci fonksiyon kurulur; tek elemanlı slotlarda $m_j = 1$, $a_j = b_j = 0$ alınır (örneğin 61 alt tablonun 0. gözüne yazılır). Derste ilk gösterilen hazır örnekte ($p = 101$, $a = 3$, $b = 42$, $m = 9$; anahtarlar 10, 22, 37, 40, 52, 60, 70, 72, 75) 10 slot 0'a düşer; 60, 72, 75 slot

2'de çakışır ve $m = 9$, $a = 10$, $b = 18$ olan ikinci fonksiyonla sırasıyla 3, 4, 7 numaralı gözlere yerleşir (ekrandan, 01:15:05 anlatımı; değerler python ile doğrulandı).

- Toplar ve kutular teoremi $m_j = n_j^2$ seçiminin gerekçesidir: kare boyutlu tabloda beklenen çakışma sayısı $1/2$'den küçük olduğundan çakışmasız bir fonksiyon birkaç denemede bulunur. İkinci teorem ise toplam alanın doğrusal kaldığını gösterir. İspatta $\sum n_j = n$ ve $\sum \binom{n_j}{2}$'nin beklenen değeri, tüm çiftlerin çakışma sayısı olan $\binom{n}{2}\frac{1}{m}$'dir; $m = n$ konunca $2n - 1$ bulunur.
- Cuckoo örneğinde sırayla: A tablo 1'in 0. gözüne; B aynı göze gelir, A tablo 2'nin 2. gözüne atılır; C tablo 1'in 1. gözüne; D aynı göze gelir, C tablo 2'nin 4. gözüne atılır; E tablo 1'in 3. gözüne; F aynı göze gelir, E tablo 2'nin 2. gözüne atılır, oradaki A tablo 1'in 0. gözüne döner, oradaki B tablo 2'nin 0. gözüne gider. G (1, 2) eklenince D, B, A, E, F, C sırayla yer değiştirir; C, G'yi tablo 1'in 1. gözünden atar ve aynı elemanlar aynı slotlar arasında dönmeye başlar. Bu kısır döngü yeniden hashing gerektiğini gösterir (döngü python ile doğrulandı).
- Hopscotch hashingde arama yalnız komşulukta yapıldığı için $O(1)$'dir. Eklemede boşluk geri taşınırken taşınacak elemanın yeni yerinin kendi ev slotunun komşuluğunda kalması gerekir. Hop bitmap'leri bu kontrolü kolaylaştırır: bir slotun bitmap'i, o slota ait elemanların hangi komşu slotlarda durduğunu gösterir. İkinci örnekte ev slotu 6 olan I eklenirken boşluk 14'ten sırasıyla G, F ve B taşınarak 9'a kadar geri getirildi; I 9'a yerleşti ve slot 6'nın hop dizisi 1001 oldu.
- Genişletilebilir hashingde bölünen kova yalnızca kendi bloğunu etkiler; diğer kovalar diskten okunmaz, yalnız dizin güncellenir. Kovalar çok küçük seçilirse yapı çok genişler; bu durumda kovalarda değer yerine işaretçi tutmak düşünülebilir.
-
-
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.